OPERATING SYSTEMS – CS23431

EXPERIMENT 01

1.(A) First come first serve Scheduling Algorithm

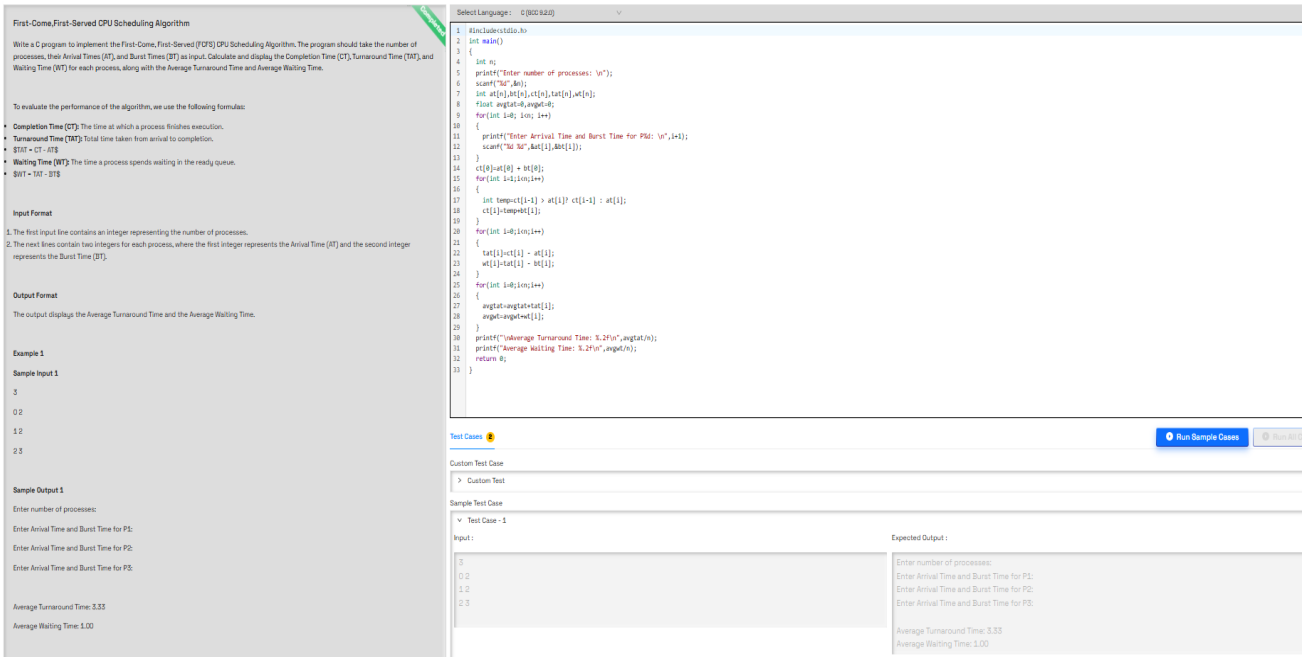


FIGURE: 1.1

1.(b) Shortest Job First Scheduling Algorithm

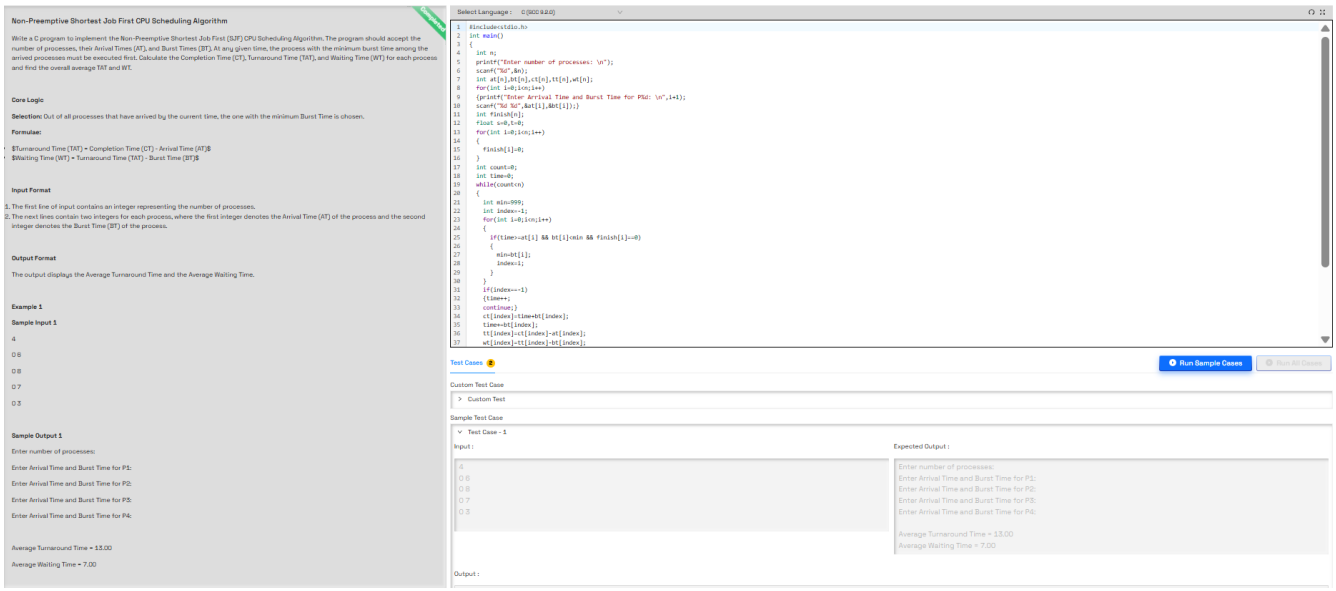
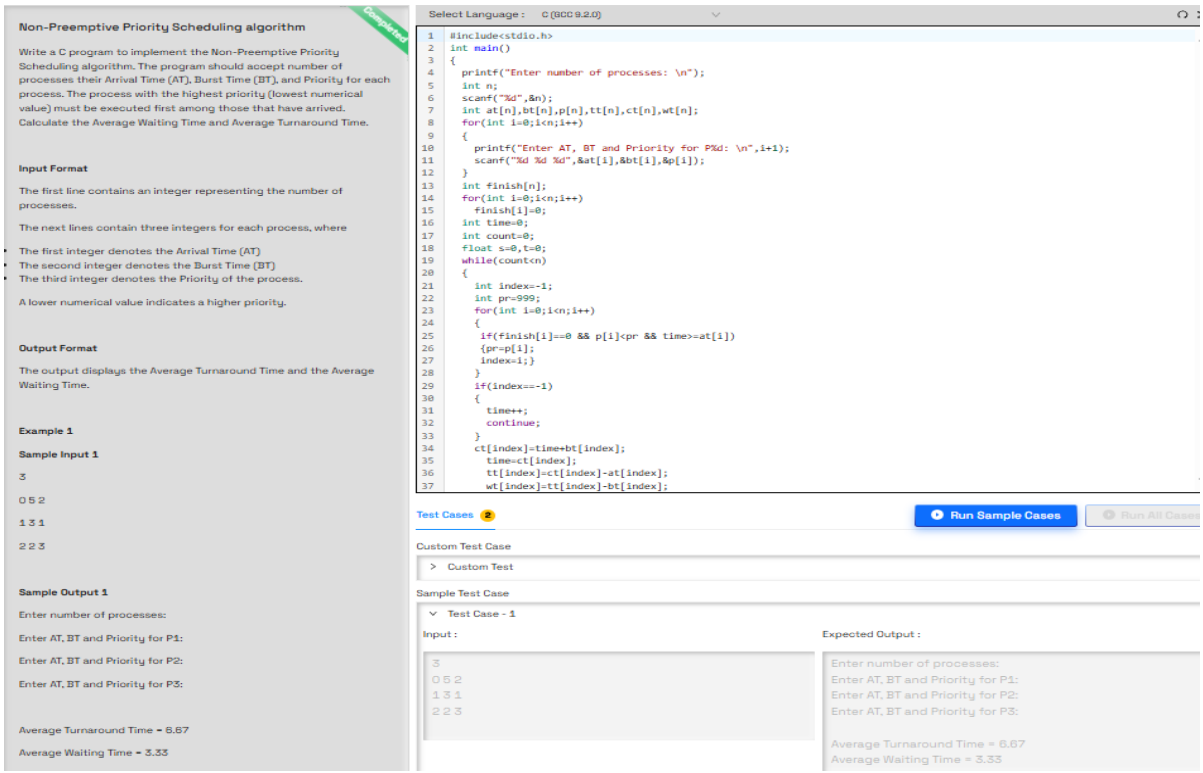


FIGURE: 1.2

1.(c) Non-Preemptive Priority Scheduling Algorithm



Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
3
0 5 2
1 3 1
2 2 3
```

Sample Output 1

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

```c
#include<stdio.h>
int main()
{
  printf("Enter number of processes: \n");
  int n;
  scanf("%d",&n);
  int at[n],bt[n],p[n],tt[n],ct[n],wt[n];
  for(int i=0;i<n;i++)
  {
    printf("Enter AT, BT and Priority for P%d: \n",i+1);
    scanf("%d %d %d",&at[i],&bt[i],&p[i]);
  }
  int finish[n];
  for(int i=0;i<n;i++)
    finish[i]=0;
  int time=0;
  int count=0;
  float s=0,t=0;
  while(count<n)
  {
    int index=-1;
    int pr=999;
    for(int i=0;i<n;i++)
    {
     if(finish[i]==0 && p[i]<pr && time>=at[i])
     {pr=p[i];
     index=i;}
    }
    if(index==-1)
    {
      time++;
      continue;
    }
    ct[index]=time+bt[index];
      time=ct[index];
      tt[index]=ct[index]-at[index];
      wt[index]=tt[index]-bt[index];
```

Test Cases 2 — Run Sample Cases

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input :

```
3
0 5 2
1 3 1
2 2 3
```

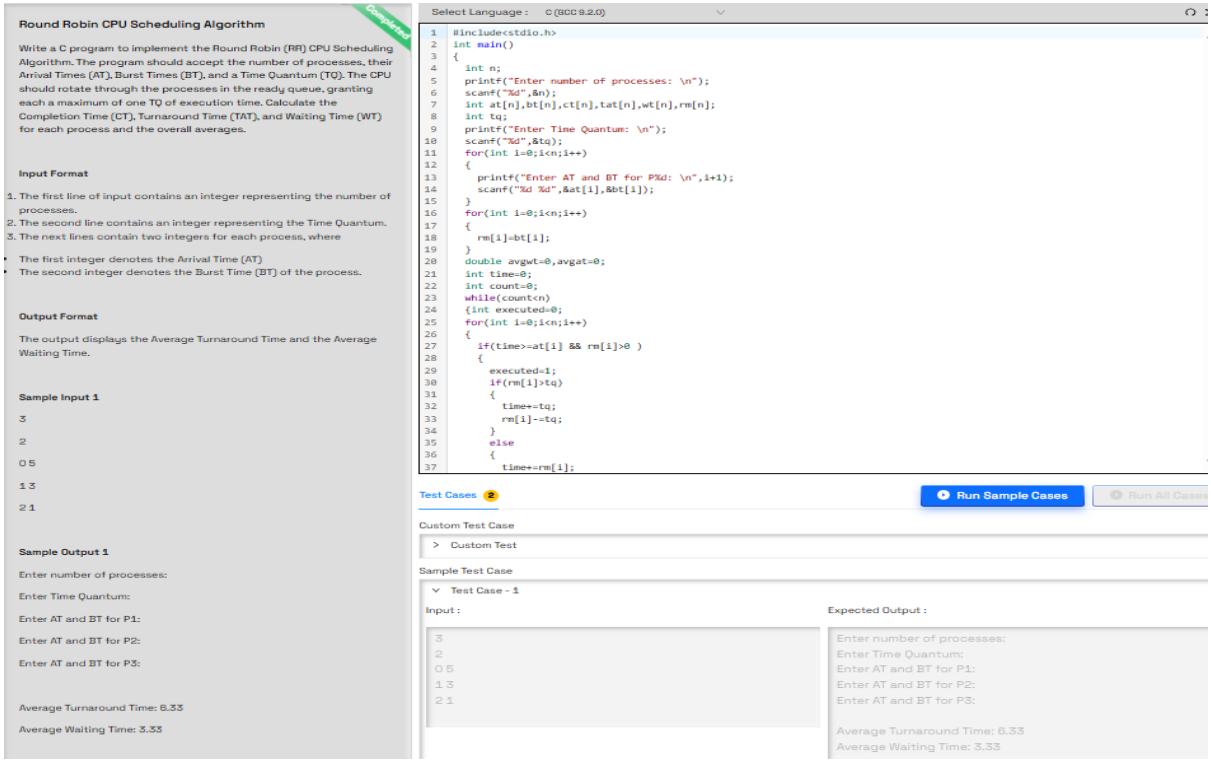
Expected Output :

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

FIGURE: 1.3

1.(d) Round Robin Scheduling Algorithm



Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next lines contain two integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

```
3
2
0 5
1 3
2 1
```

Sample Output 1

```
Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:

Average Turnaround Time: 6.33
Average Waiting Time: 3.33
```

```c
#include<stdio.h>
int main()
{
  int n;
  printf("Enter number of processes: \n");
  scanf("%d",&n);
  int at[n],bt[n],ct[n],tat[n],wt[n],rm[n];
  int tq;
  printf("Enter Time Quantum: \n");
  scanf("%d",&tq);
  for(int i=0;i<n;i++)
  {
    printf("Enter AT and BT for P%d: \n",i+1);
    scanf("%d %d",&at[i],&bt[i]);
  }
  for(int i=0;i<n;i++)
  {
    rm[i]=bt[i];
  }
  double avgwt=0,avgat=0;
  int time=0;
  int count=0;
  while(count<n)
  {int executed=0;
  for(int i=0;i<n;i++)
  {
    if(time>=at[i] && rm[i]>0 )
    {
      executed=1;
      if(rm[i]>tq)
      {
        time+=tq;
        rm[i]-=tq;
      }
      else
      {
        time+=rm[i];
```

Test Cases 2 — Run Sample Cases

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input :

```
3
2
0 5
1 3
2 1
```

Expected Output :

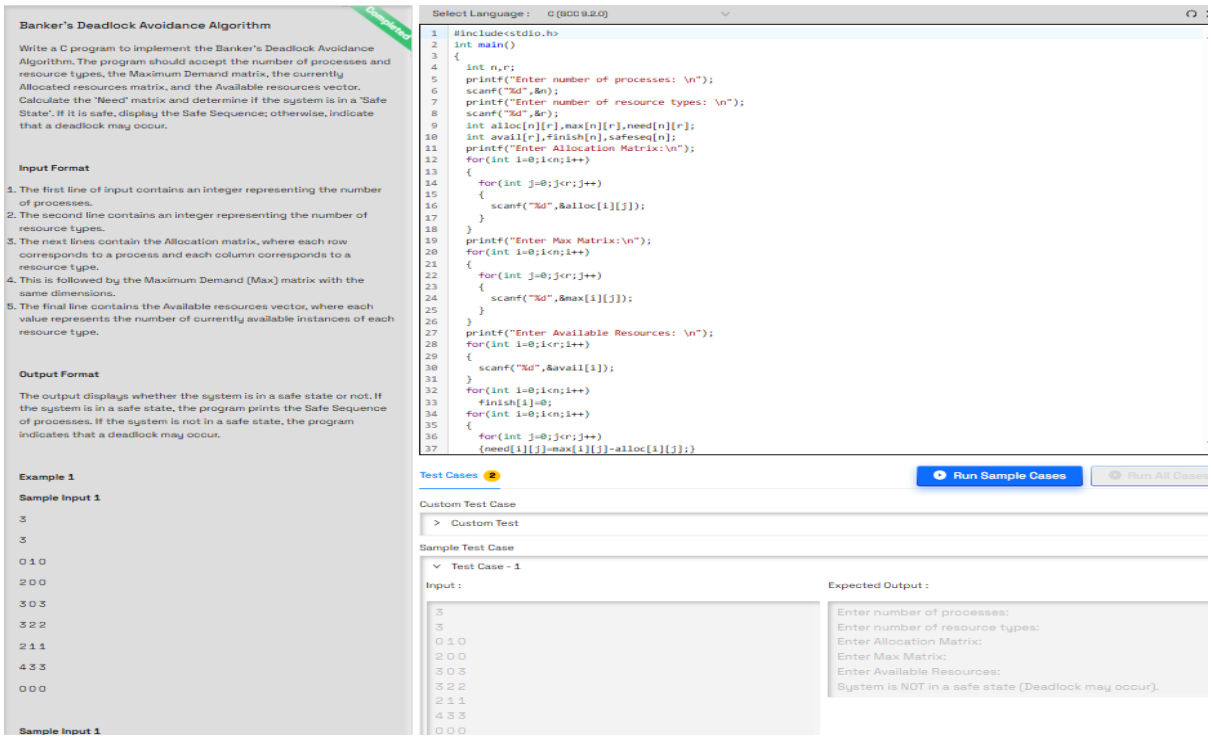
```
Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:

Average Turnaround Time: 6.33
Average Waiting Time: 3.33
```

FIGURE: 1.4

EXPERIMENT 02

2) Banker's Deadlock Avoidance Algorithm



Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Output Format

The output displays whether the system is in a safe state or not. If the system is in a safe state, the program prints the Safe Sequence of processes. If the system is not in a safe state, the program indicates that a deadlock may occur.

Example 1

Sample Input 1

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Sample Input 1

```c
#include<stdio.h>
int main()
{
  int n,r;
  printf("Enter number of processes: \n");
  scanf("%d",&n);
  printf("Enter number of resource types: \n");
  scanf("%d",&r);
  int alloc[n][r],max[n][r],need[n][r];
  int avail[r],finish[n],safeseq[n];
  printf("Enter Allocation Matrix:\n");
  for(int i=0;i<n;i++)
  {
    for(int j=0;j<r;j++)
    {
      scanf("%d",&alloc[i][j]);
    }
  }
  printf("Enter Max Matrix:\n");
  for(int i=0;i<n;i++)
  {
    for(int j=0;j<r;j++)
    {
      scanf("%d",&max[i][j]);
    }
  }
  printf("Enter Available Resources: \n");
  for(int i=0;i<r;i++)
  {
    scanf("%d",&avail[i]);
  }
  for(int i=0;i<n;i++)
    finish[i]=0;
  for(int i=0;i<n;i++)
  {
    for(int j=0;j<r;j++)
    {need[i][j]=max[i][j]-alloc[i][j];}
```

Test Cases 2 — Run Sample Cases

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input :

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

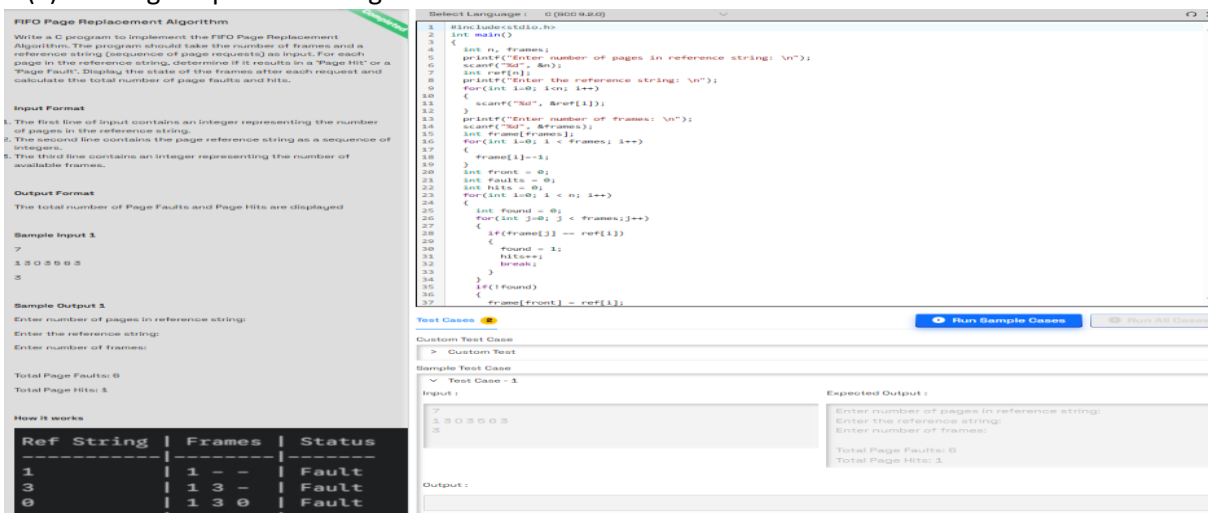
Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

FIGURE: 2.0

EXPERIMENT 03

3.(a)FIFO Page Replacement Algorithm



FIFO Page Replacement Algorithm

Write a C program to implement the FIFO Page Replacement Algorithm. The program should take the number of frames and a reference string (sequence of page requests) as input. For each page in the reference string, determine if it results in a "Page Hit" or a "Page Fault". Display the state of the frames after each request and calculate the total number of page faults and hits.

Input Format

1. The first line of input contains an integer representing the number of pages in the reference string.
2. The second line contains the page reference string as a sequence of integers.
3. The third line contains an integer representing the number of available frames.

Output Format

The total number of Page Faults and Page Hits are displayed

Sample Input 1

```
7
1 3 0 3 5 6 3
3
```

Sample Output 1

```
Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 6
Total Page Hits: 1
```

How it works

Ref String	Frames	Status
1	1 - -	Fault
3	1 3 -	Fault
0	1 3 0	Fault

```c
#include<stdio.h>
int main()
{
  int n, frames;
  printf("Enter number of pages in reference string: \n");
  scanf("%d", &n);
  int ref[n];
  printf("Enter the reference string: \n");
  for(int i=0; i<n; i++)
  {
    scanf("%d", &ref[i]);
  }
  printf("Enter number of frames: \n");
  scanf("%d", &frames);
  int frame[frames];
  for(int i=0; i < frames; i++)
  {
    frame[i]=-1;
  }
  int front = 0;
  int faults = 0;
  int hits = 0;
  for(int i=0; i < n; i++)
  {
    int found = 0;
    for(int j=0; j < frames;j++)
    {
      if(frame[j] == ref[i])
      {
        found = 1;
        hits++;
        break;
      }
    }
    if(!found)
    {
      frame[front] = ref[i];
```

Test Cases 8 — Run Sample Cases

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input :

```
7
1 3 0 3 5 6 3
3
```

Expected Output :

```
Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 6
Total Page Hits: 1
```

Output :

FIGURE:

3.(b) LRU Page Replacement Algorithm

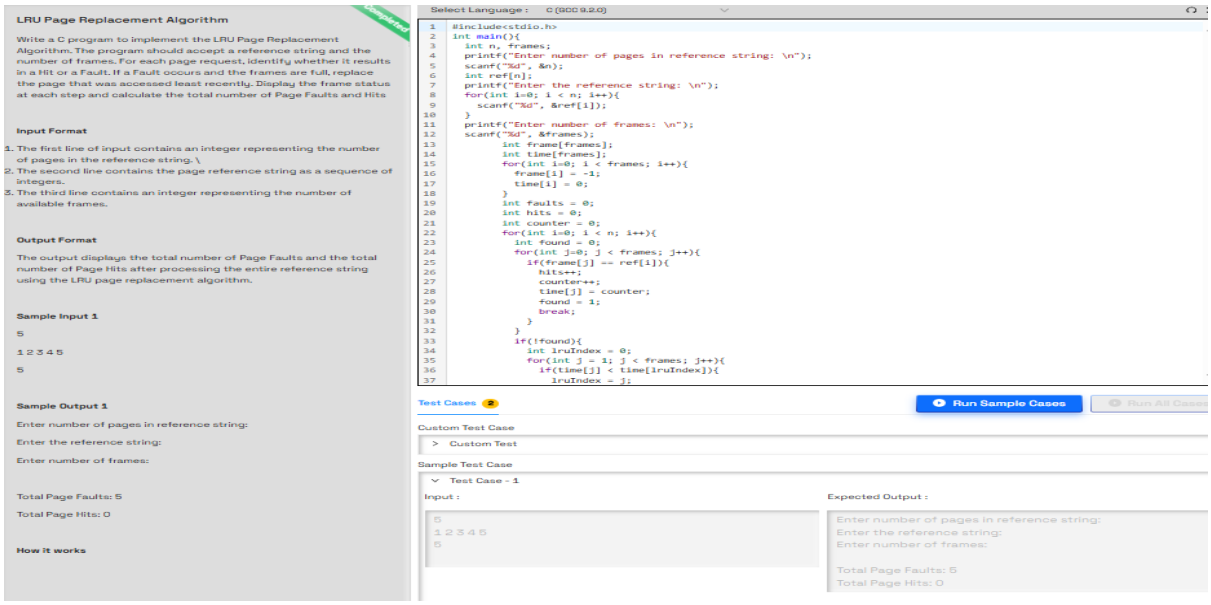


FIGURE: 3.2

EXPERIMENT 04

4.(a)First Fit Contigous Memory Allocation

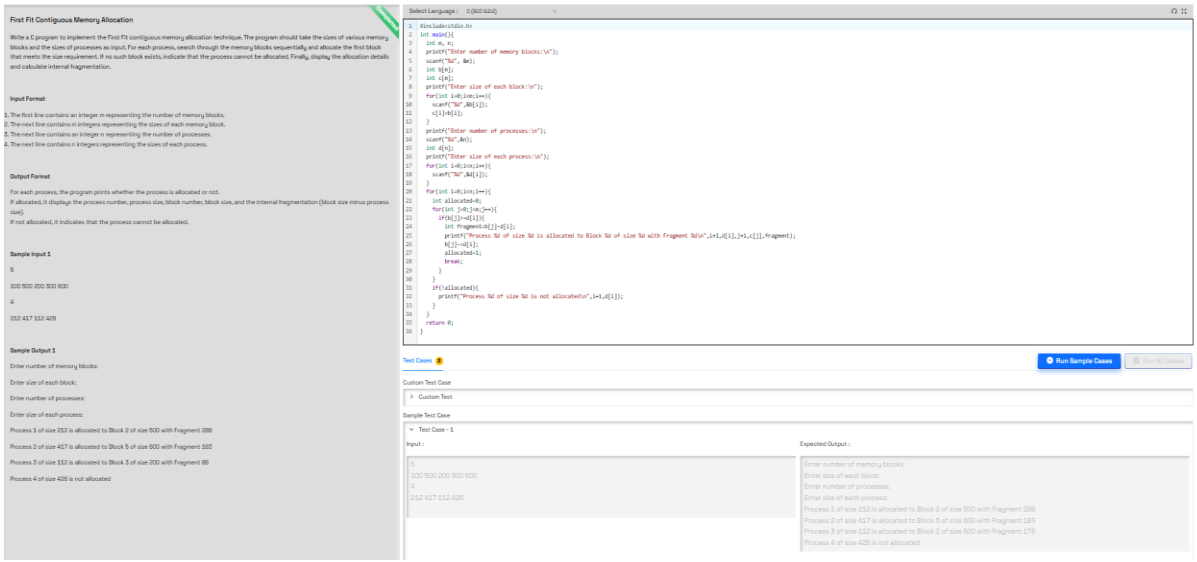


FIGURE: 4.1

4.(b) Best Fit Contigous Memory Allocation

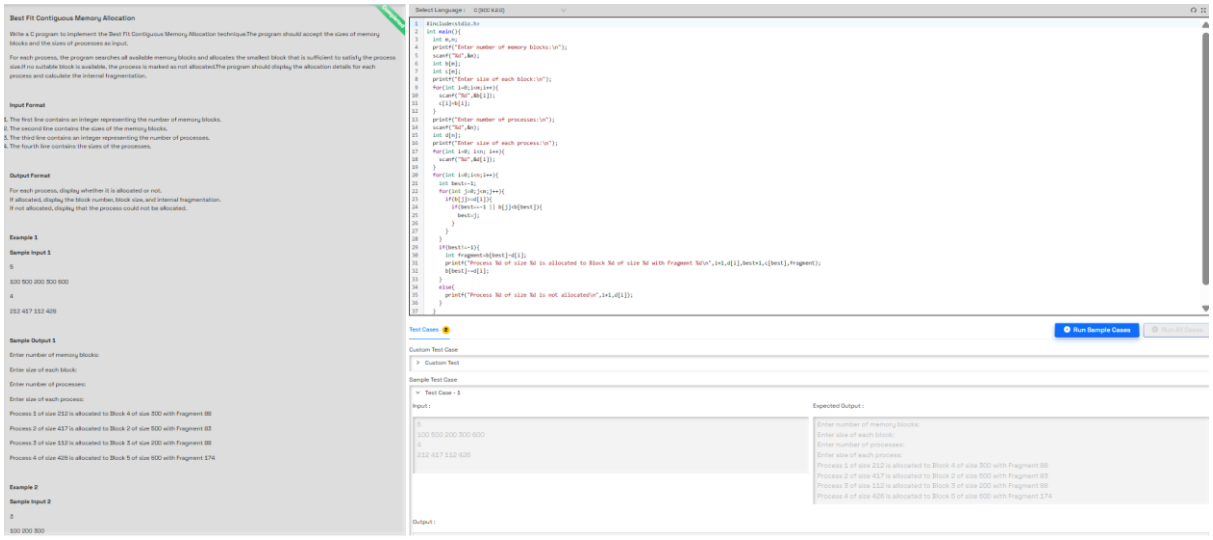


FIGURE: 4.2